For **Phase 1**, I'd treat this like a forensic investigation rather than a migration activity. The objective is to reconstruct the **original intended execution model** using evidence from the Unisys manuals, not assumptions from the converted PowerShell.

Phase 1 Manual Research Checklist

Recover the Original SSG / ECL Execution Model

Research Output Template

Create a spreadsheet or markdown table with:

Topic	Evidence Found	Manual	Page	Confidence
XQT command				
UCOB				
MAP				
ACOB				
SSG OUTPUT statement				

Do not write conclusions without a page reference.

Section 1 - Understand What SSG Actually Is

Locate

Search all manuals for:

```
SSG
Symbiont Stream Generator
Symbiont Generator
Generator Program
Program Generator
```

Questions to Answer

Q1

Is SSG described as:

- a programming language
- a macro language
- a report writer
- a code generator
- a job control facility

Q2

What are the normal outputs of an SSG run?

Possible findings:

```
COBOL source
RPG source
Report definitions
Job streams
Printer files
Other
```

Q3

What is the stated lifecycle?

Diagram the sequence:

```
SSG Source
 ↓
???
 ↓
???
 ↓
Final Result
```

Deliverable

One-page summary:

```
What SSG is
What SSG generates
How SSG-generated artefacts are used
```

Section 2 - Understand External Program Invocation

Search Terms

```
XQT
@XQT
EXECUTE
RUN
External Program
CALL PROGRAM
```

```
7  Program Invocation
```

Questions

Q4

What does:

```
1  @XQT
```

actually do?

Q5

Can XQT invoke:

```
1  compiled programs
2  compiler utilities
3  system utilities
4  SSG programs
```

Q6

How are parameters passed?

Examples:

```
1  @XQT UCOB
2  @XQT MAP
3  @XQT ACOB
```

record:

- syntax
- parameter rules
- examples

Q7

Does XQT wait for completion?

Determine whether execution is:

```
1  synchronous
2  asynchronous
3  batch queued
```

Deliverable

Document:

```
1  XQT Execution Model
```

Section 3 - Research UCOB

Search Terms

```
1  UCOB
2  COBOL Compiler
3  Compile COBOL
4  COBOL Generation
```

Questions

Q8

Exactly what is UCOB?

Possible outcomes:

```
1  Compiler
2  Preprocessor
3  Linker
4  Executor
5  Migration Utility
```

Q9

Input artefacts?

Examples:

```
1  COBOL source
2  Generated source
3  Object code
```

Q10

Output artefacts?

Examples:

```
1  Object
2  Executable
3  Library
4  Run unit
```

Q11

Typical JCL/ECL example?

Capture a complete example.

Deliverable

One-page UCOB profile.

Section 4 - Research MAP

Search Terms

```
MAP
MAP utility
MAP processor
```

Questions

Q12

What is MAP?

Q13

Where does MAP sit?

```
Before UCOB?
After UCOB?
Independent?
```

Q14

Is MAP:

```
Compilation
Linking
Deployment
Execution
```

Deliverable

Document the complete MAP lifecycle.

Section 5 - Research ACOB

Search Terms

```
ACOB
COBOL execution
ACOB utility
```

Questions

Q15

What exactly is ACOB?

Q16

Does ACOB execute COBOL programs?

Q17

What input does it consume?

Q18

What outputs does it produce?

Deliverable

Document the complete ACOB lifecycle.

Section 6 - Dataset Assignment Semantics

This is critical to understanding what eventually becomes:

```
GetSgsValue()
```

and

```
Call-ExternalProgram()
```

Search Terms

```
INPUT
OUTPUT
FILE
DISC
ASSIGN
```

```
6  FILE ASSIGNMENT
7  DATASET
```

Questions

Q19

How are files assigned?

Example:

```
1  INPUT ECL-TAPEIN
2  OUTPUT ECL-TAPEOUT
```

Q20

When does resolution occur?

At:

```
1  SSG generation time?
2  Compile time?
3  Execution time?
```

Q21

Can file assignments be overridden?

Deliverable

Dataset Resolution Flow Diagram.

Section 7 - Symbiont Variables and Parameter Passing

Search Terms

```
1  SET
2  *SET
3  *INCREMENT
4  Symbol
5  Variable
6  Substitution
7  Parameter
```

Questions

Q22

When are *SET variables evaluated?

Q23

Do they become:

```
1  Generated source text
2  Runtime variables
3  Both
```

Q24

How are values propagated to external programs?

This is directly related to the future AMT implementation.

Deliverable

Variable Propagation Diagram.

Section 8 - Determine the End-to-End Chain

After completing Sections 1-7, answer these questions with evidence.

Final Questions

Q25

Is SSG intended to generate COBOL?

Q26

What role does UCOB play?

Q27

What role does MAP play?

Q28

What role does ACOB play?

Q29

Which stage first performs real file I/O?

Is it:

```
SSG
Generated COBOL
Compiled COBOL
Executed COBOL
```

Q30

At what point should a file such as:

```
ECL-FICFORMAT
ECL-TAPEOUT
ECL-FICARCHI
```

be physically created?

Expected Outcome

At the end of Phase 1 you should be able to produce a single evidence-backed diagram similar to:

```
SSG Source
    ↓
SSG Runtime
    ↓
Generated COBOL
    ↓
UCOB
    ↓
Compiled COBOL
     ↓
 MAP
     ↓
 Executable
     ↓
 ACOB
     ↓
 Runtime Execution
     ↓
 Business Output Files
```

or prove that some variation of that flow is correct.

That diagram becomes the baseline for Phases 2, 3 and 4 and should prevent the team from making further assumptions about where the actual business logic executes.